



BSc (Hons) in **Information
Technology Specialized in AI**
**IT3012 : Intelligent
Agents**

Faculty of Computing

Practical 01

Year 3 · Semester 1 · 2026 Semester
2

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width` and `self.height`: It defines the physical boundary and size of the grid
`self.walls`: They are the fixed physical obstacles within the environment.
`self.food_positions`: It stores the coordinates of all the food targets scattered through the grid.
`self.opponents`: It tracks the current physical positions of the adversarial entities in the grid.
`self.agent_pos`: It track the physical location (x, y coordinates) of the agent within the environment.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent
Because the code includes a `self.opponents` list. This means the main player isn't alone—there are other active enemies moving around the grid that the player has to dodge or compete against.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The complete history of everything the agent has perceived (received via its sensors) from the very beginning of its operation up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S)

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable.
The agent's "sensors" don't give it the coordinates of all the walls or food. It only finds out about them if it actually steps on them. Since it can't see the whole map at once, its vision is partial.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure.
The score is the metric used to judge how well the agent is doing its job.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall)

rather than the agent's internal processing effort?

We care about results (like grabbing food or not crashing), not how hard the agent's brain worked. If we scored it based on how much it "thought," it might just run in circles calculating things forever to get a high score, instead of actually playing the game.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.
9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment remains partially observable, and hiding the traps increases the uncertainty for the agent. Because the traps cannot be detected by its sensors, the agent does not have complete knowledge of the grid. It can only identify a trap after stepping on it, which means it must make decisions based on limited information.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Acting "rationally" simply means doing whatever maximizes the score. By giving the agent this new sensor, it can now learn to detect and step away from traps. Avoiding traps prevents score penalties, directly allowing the agent to make better, smarter choices that maximize its final score.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to

decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

This happens when the vacuum cleaner agent is rewarded only for cleaning dirt instead of keeping the floor clean. It may learn to dump dirt and clean it repeatedly to gain unlimited rewards without achieving the actual goal. This shows why performance measures should focus on the desired outcome, not just the agent's actions.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING